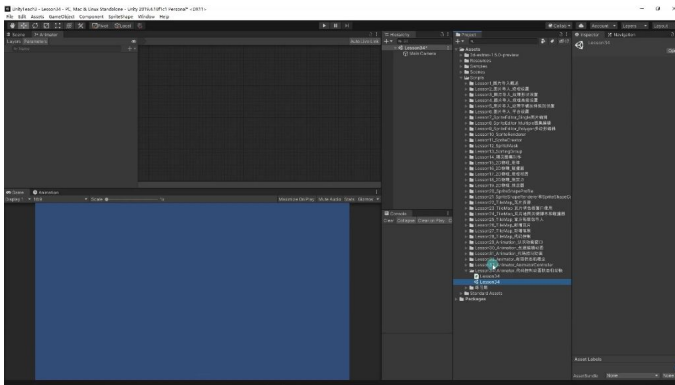
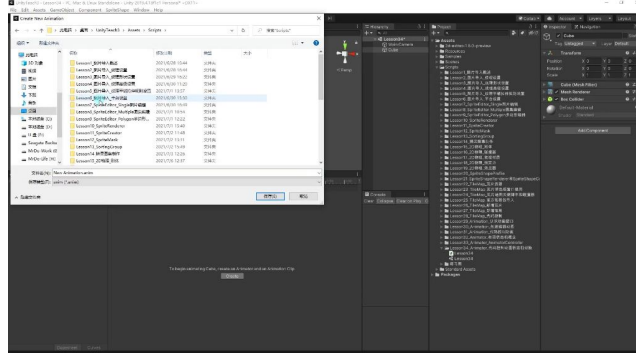


一、知识回顾 00:21

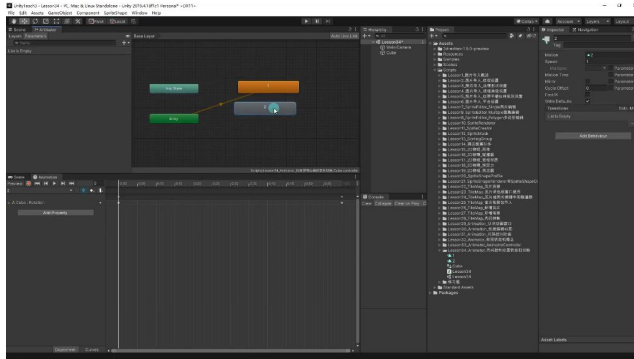
1. Animator中的API 00:32



- 创建动画步骤：
 - 新建场景并将摄像机改为透视摄像机
 - 创建立方体并重置位置为(0,0,0)
 - 打开Animation窗口并选中对象
 - 创建动画文件并保存到指定文件夹
- 自动添加组件：
 - 创建动画后系统会自动添加Animator组件
 - 同时创建状态机文件和动画文件
 - 状态机文件中动画文件作为默认状态
 - Animator组件自动关联状态机文件

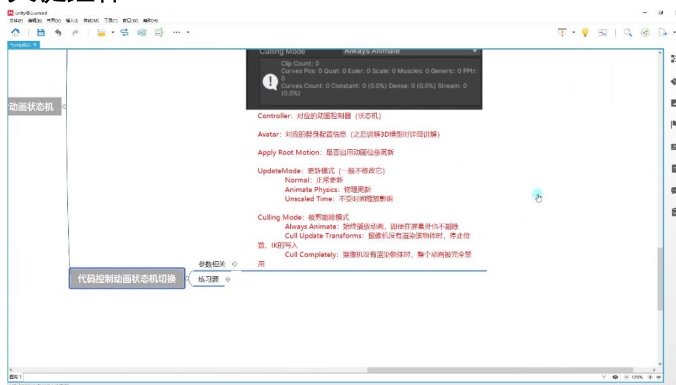


- 动画录制：
 - 位移动画：在1秒位置设置y轴位移到10
 - 旋转动画：y轴旋转90度
 - 可预览动画效果
- 状态机设置：
 - 动画状态自动添加到Animator窗口
 - 默认状态为第一个创建的动画
 - 通过右键连接状态实现切换

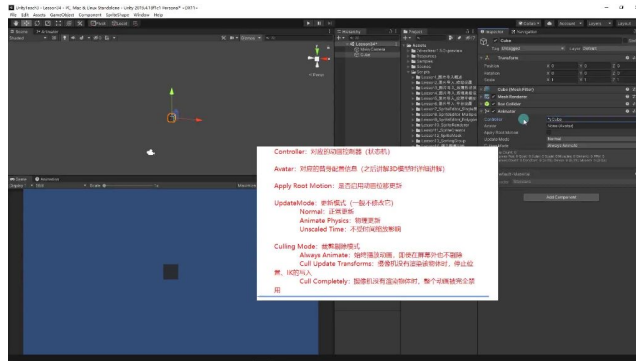


- 条件设置：
 - 可添加四种类型条件（示例使用布尔值）
 - 命名条件变量（如"change"）
 - 设置切换条件：true切换到状态2，false切回状态1
- 测试运行：
 - 默认循环播放第一个状态
 - 修改条件值可触发状态切换
 - 通过改变布尔值实现动画状态切换控制
- 核心要点：
 - 新动画系统自动处理组件关联
 - 状态连线实现动画过渡
 - 条件控制实现状态切换逻辑
 - 简单示例展示基本工作原理

二、关键组件Animator



- 1. 核心参数解析
 - **Controller:** 关联动画控制器文件(.controller)，用于控制动画状态机的切换逻辑。示例中默认关联的是自动创建的cube状态机文件。
 - **Avatar配置:** 3D模型专用的替身骨骼配置信息，需配合人形动画使用（将在3D模型专题详细讲解）。
 - **根运动控制:**
 - **Apply Root Motion:** 启用时动画位移会实时更新Transform组件位置（对3D角色移动特别重要）
 - 典型应用场景：角色行走动画自带位移时，需勾选此项使模型实际移动

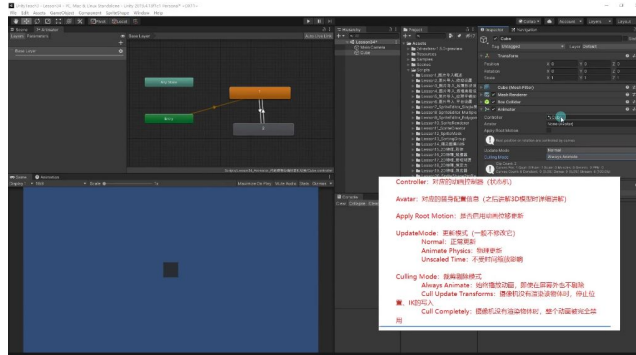


2. 更新模式配置

- **Normal模式:** 默认选项, 按常规时间轴更新动画
- **物理更新模式:**
 - **Animate Physics:** 动画更新与物理系统同步 (适用于物理交互场景)
 - 典型案例: 被击飞的角色动画需要与物理碰撞同步时使用
- **独立时间模式:**
 - **Unscaled Time:** 不受Time.timeScale影响 (适用于游戏暂停时的UI动画)

3. 裁剪优化策略

- **Always Animate:**
 - 强制持续播放动画 (性能消耗最高)
 - 适用场景: 关键NPC/主角等必须保证动画流畅的对象
- **Cull Update Transforms:**
 - 屏幕外时停止IK和位移计算 (平衡性能与效果)
 - 技术细节: 仍保留基础动画状态但停止物理计算
- **完全禁用模式:**
 - **Cull Completely:** 超出视口立即停止所有动画运算
 - 适用场景: 大量背景装饰物等次要动画元素

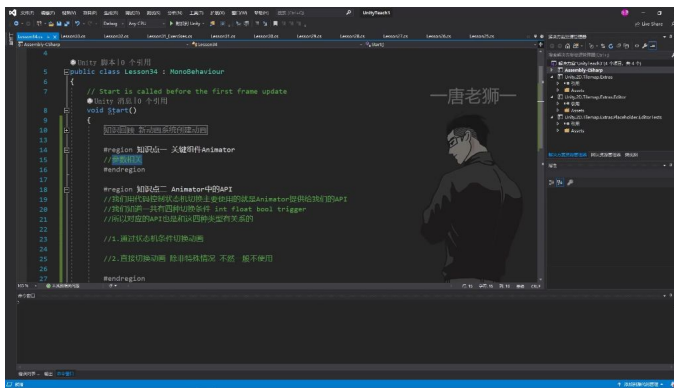


4. 开发注意事项

- **Controller必选性:** 唯一必须配置的参数, 其他参数均有默认值
- **3D专用参数:** Avatar和Root Motion参数需结合后续3D课程理解
- **性能调优原则:** 根据对象重要性选择Culling Mode, 通常建议:
 - 主角使用Always Animate
 - 次要NPC使用Cull Update Transforms
 - 环境物体使用Cull Completely

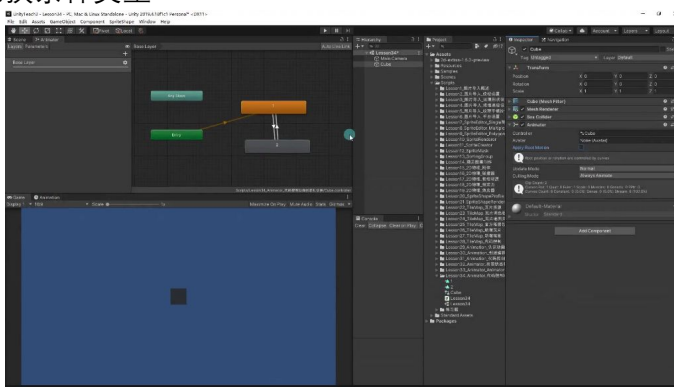
三、Animator中的API 06:53

1. Animator API概述 06:57



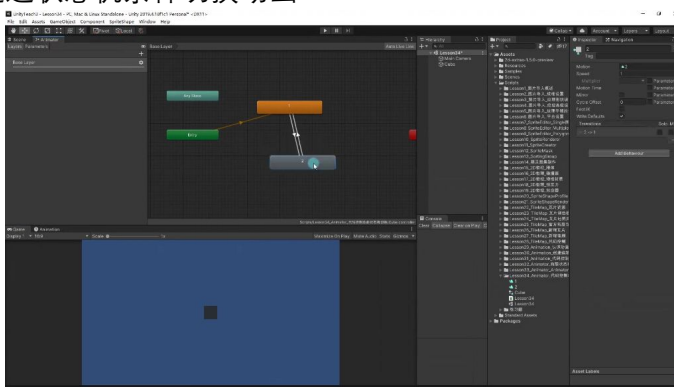
-
- **核心功能：**通过代码控制状态机切换，主要使用Animator提供的API
- **设计思想：**基于有限状态机(FSM)概念，状态切换需满足特定条件
- **使用场景：**新动画系统中替代老动画系统的直接播放动画方式

2. 切换条件类型 07:06



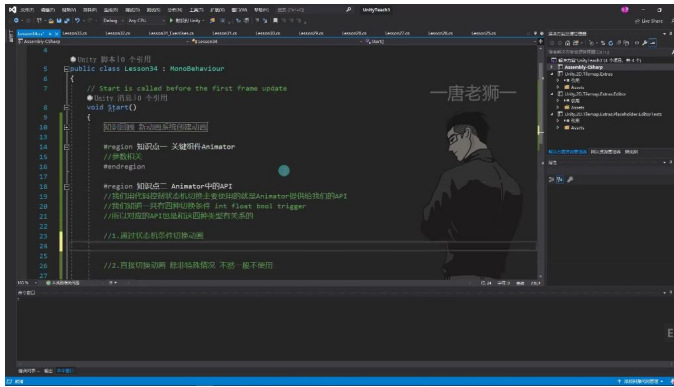
-
- **四种类型：**
 - int：整型参数
 - float：浮点型参数
 - bool：布尔型参数
 - trigger：触发器参数
- **对应关系：**API设计与这四种参数类型完全对应

3. 通过状态机条件切换动画 07:18



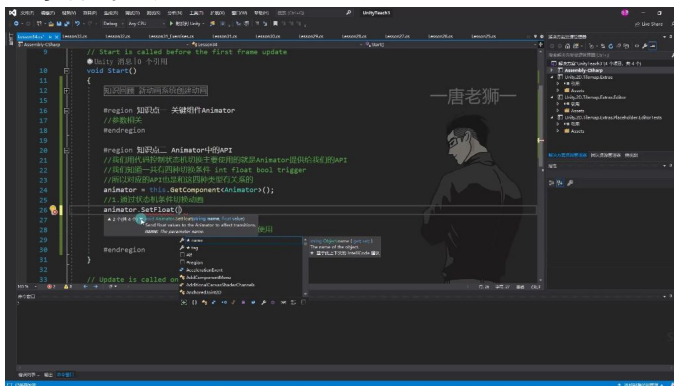
-
- **基本原理：**通过改变条件值触发状态转换
- **条件检测：**状态机自动检测条件是否满足
- **退出时间：**注意Has Exit Time设置会影响切换即时性

4. 获取Animator组件 07:59



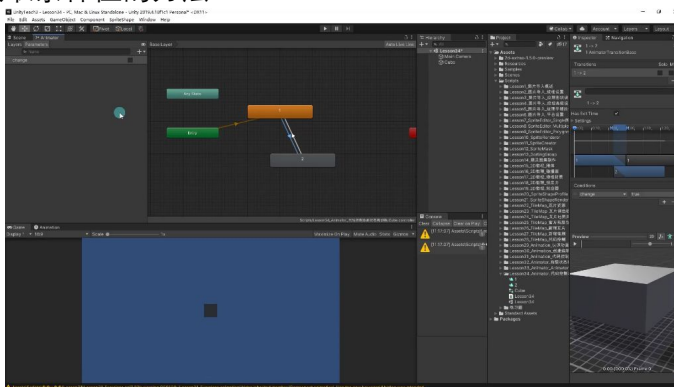
-
- 声明方式: private Animator animator;
- 命名空间: 属于UnityEngine命名空间, 无需额外引用
- 获取方法: animator = GetComponent<Animator>();

5. 设置条件的方法 08:43



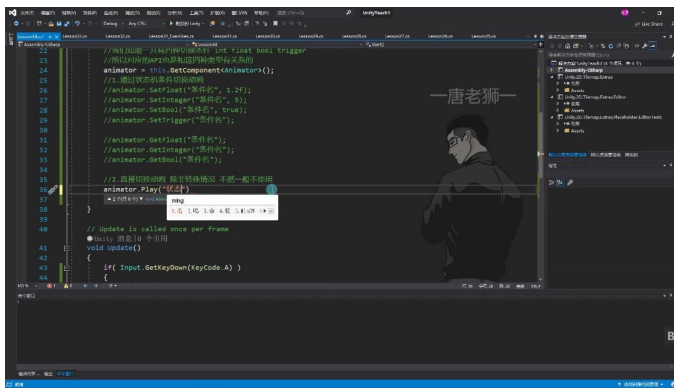
-
- **Float类型**:.SetFloat("条件名", 1.2f)
- **Int类型**: SetInteger("条件名", 5)
- **Bool类型**: SetBool("条件名", true)
- **Trigger类型**: SetTrigger("条件名")
- 参数说明: 第一个参数为条件名称字符串, 第二个为要设置的值

6. 获取条件值的方法 10:30



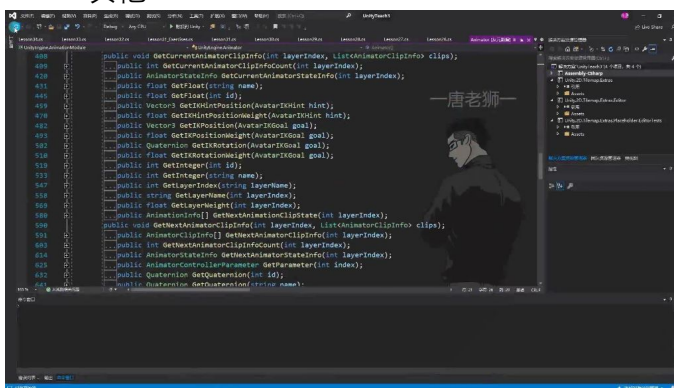
-
- **Float类型**: GetFloat("条件名")
- **Int类型**: GetInteger("条件名")
- **Bool类型**: GetBool("条件名")
- 使用场景: 可用于条件判断和逻辑处理

7. 直接切换动画的方法 15:57



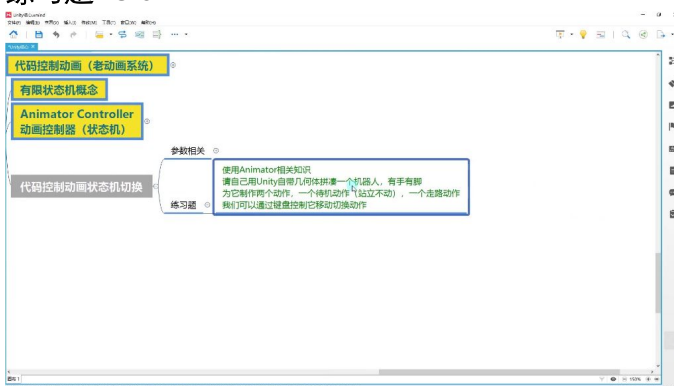
-
- 方法: Play("状态名")
- 注意事项:
 - 填写的是状态名而非动画名
 - 特殊情况才使用, 常规推荐通过条件切换
 - 会绕过状态机条件检测机制

8. Animator其他API 17:15



-
- 扩展功能: 包含IK控制、层级权重等高级功能
- 学习建议: 先掌握基础条件控制API, 高级功能后续再学
- 实际应用: 3D模型动画控制会涉及更多复杂API

四、练习题 18:01



-
- 任务目标: 制作可切换动作的机器人
- 具体要求:
 - 使用Unity基本几何体搭建机器人模型
 - 创建待机和行走两个动画状态
 - 实现键盘控制移动和动作切换
- 技术要点:
 - 状态机条件设置
 - 动画过渡参数调整

- 脚本控制逻辑实现

五、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
Animator动画状态机切换	通过代码控制动画状态切换，使用 SetBool、SetFloat、SetInteger、SetTrigger等API改变条件触发状态转换	条件类型选择 (Bool/Float/Int/Trigger) 与 连线配置 (Has Exit Time的影响)	★ ★
Animator组件参数	Controller关联状态机文件、Update Mode (Normal/Unscaled Time/Animate Physics)、Culling Mode (动画裁剪策略)	替身配置 (Avatar) 与 位移更新 (Apply Root Motion) 需结合3D模型讲解	★
状态机连线逻辑	状态间通过条件连线切换，可设置Has Exit Time控制是否等待当前动画播放完毕	条件优先级与多条件组合 的复杂逻辑 (未展开)	★ ★
API方法对比	SetBool/GetBool vs SetTrigger: 前者需显式设置值，后者单次触发后自动重置	Trigger 的自动复位特性易被忽略	★ ★
新动画系统设计思想	有限状态机 (FSM) : 通过条件驱动状态切换，而非直接播放动画 (与旧版 Animation组件差异)	状态与动画解耦 (状态名 ≠ 动画名)	★ ★